
EMPLOYMENT MANAGEMENT SYSTEM

End-to-End Round Trips

Every flow traced in both directions, browser to database and back

FULL REQUEST AND RESPONSE TRACES • 2 OCTOBER 2026

Contents

01	1. How to read a trace	3
02	2. Trace 1 - Log in	4
03	3. Trace 2 - Register	6
04	4. Trace 3 - Session restore on reload	7
05	5. Trace 4 - Log out	8
06	6. Trace 5 - Browse the employee list	9
07	7. Trace 6 - Search employees by name	10
08	8. Trace 7 - Create an employee	11
09	9. Trace 8 - Update an employee	12
10	10. Trace 9 - Delete an employee	13
11	11. Trace 10 - Browse departments	14
12	12. Trace 11 - Create / update / delete a department	14
13	13. Trace 12 - View a department with its employees	15
14	14. Trace 13 - Check in	16
15	15. Trace 14 - Check out	18
16	16. Trace 15 - Manager searches attendance	19
17	17. Trace 16 - An employee views their own attendance	20
18	18. Trace 17 - The dashboard	22
19	19. Trace 18 - A request that is refused	23
20	20. Every trace at a glance	24
21	21. The five questions to ask about any flow	24

Every flow in this application, traced in **both directions**: what the browser does and sends, what each backend layer does with it, and exactly how the response travels back to become pixels on screen.

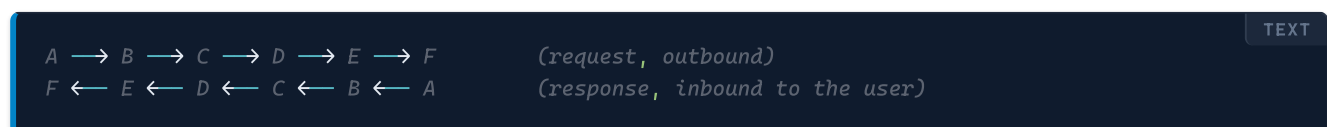
Read this document when you want to explain a flow start to finish. Read `01-foundations.md` first for the layer vocabulary.

1. How to read a trace

Every trace below has the same six stages. Learn the shape once and you can explain any flow in the app:

Stage	Where it happens	What you are looking at
A. Trigger	Browser	the click, the state change, the decision to fetch
B. Request	<code>api/http.js</code>	the URL, the method, the headers, the body
C. Inbound	Servlet + security	the filter chain, authentication, <code>@PreAuthorize</code>
D. Business	Controller → Service → Repository	mapping, validation, rules, SQL
E. Response	DTO → JSON	the exact status code and body
F. Return	Browser	cache update, re-render, toast, navigation

The direction arrows matter:



Two paths carry data backwards that you should always name explicitly:

- **the JSON body** - `ErrorResponse` / `Page` / DTO, parsed by `api()`;
- **the side effects** - query-cache invalidation, `pushToast`, `navigate`, and `localStorage`.

Most "the UI did not update" bugs are a failure in stage F, not stage D.

The layer crossing table

Used throughout this document:

Crossing	Method that performs it
browser → controller	<code>fetch()</code> in <code>api/http.js</code>
controller → service	injected interface (<code>AuthService</code> , <code>EmployeeService</code> , ...)
service → repository	<code>userRepository</code> , <code>employeeRepository</code> , ...

Crossing	Method that performs it
repository → DB	Spring Data / Hibernate
entity → JSON	<code>*Mapper</code> (MapStruct) + Jackson
JSON → UI	<code>useQuery</code> / <code>useMutation</code> + React re-render

2. Trace 1 - Log in

Outbound: browser → database

A. TRIGGER

user types into `<input value={username}> / <input type="password">`
 → setUsername / setPassword → local component state
 → clicks "Sign in" → `<form onSubmit={handleSubmit}>`
 → `e.preventDefault()` (stops the browser's own form navigation)

B. REQUEST

LoginPage.jsx:26 → http.js:112

`loginUser({ username, password })`
 → `api("/api/auth/login", { method:"POST", body: JSON.stringify({ ... }) })`
 → `token = useAuthStore.getState().token` (null on first login)
 → `headers = { "Content-Type": "application/json" }`
 (Authorization omitted: there is no token yet)
 → `POST /api/auth/login { "username":"jane", "password":"secret123" }`
 → Vite dev proxy forwards to `http://localhost:8080`

C. INBOUND

SecurityConfig

Tomcat accepts the request
 → `.csrf(disable)` no cookies, so nothing to protect
 → `JwtAuthenticationFilter.doFilterInternal`
 `authHeader == null` → continue anonymous (line 29-32)
 → URL check: `"/api/auth/**"` matches → `permitAll` (line 35)
 → controller method runs

D. BUSINESS

`AuthController.login(@Valid @RequestBody LoginRequest)` (line 27)
 → Jackson deserialises JSON into `LoginRequest`
 → `@Valid`: no constraints on this DTO, nothing to check
 → `authService.login(loginRequest)`
 `AuthServiceImp.login` (line 33)
 → `authenticationManager.authenticate(`
 `new UsernamePasswordAuthenticationToken(u, p))`
 Spring's `DaoAuthenticationProvider`:
 → `CustomUserDetailsService.loadUserByUsername("jane")`
 → `userRepository.findByUsernameWithRoles("jane")`
 → `SELECT ... FROM users u LEFT JOIN FETCH u.roles`
 → `WHERE u.username = 'jane'`
 → User implements `UserDetails`, so the entity IS the principal
 → `BCryptPasswordEncoder.matches("secret123", storedHash)`
 → if the hash does not match: `BadCredentialsException`
 → `UserDetails userDetails = (UserDetails) auth.getPrincipal()`

```

→ jwtService.generateToken(userDetails)                                JwtService:32
  → header { "alg":"HS256" }
  → payload { "sub":"jane",
              "roles":["ROLE_MANAGER"],
              "iat": ... , "exp":now+864000000 }
  → Decoders.BASE64.decode(jwt.secret) → HMAC key
  → signWith(key, HS256) → header.payload.signature
→ new AuthResponse(token)

```

E. RESPONSE

200 OK

```
{ "token": "eyJhbGciOiJIUzI1NiJ9..." }
```

On failure instead: 401 { "message":"Invalid username or password" }

Inbound: response → screen

F. RETURN

LoginPage.jsx:26-34

TEXT

```

api() sees response.ok === true
→ response.json() → { token }
→ setSession(token)
  localStorage.setItem("ems_token", token)
  set({ token })                                Zustand notifies subscribers
→ await loadUser()
  → fetch("/api/auth/me", { Authorization: "Bearer <token>" }) (a SECOND request)
  → backend: JwtAuthenticationFilter now FINDS the header
    → jwtService.extractUsername(token) verifies signature + expiry
    → customUserDetailsService.loadUserByUsername("jane") DB hit again
    → jwtService.isTokenValid(token, userDetails)
    → SecurityContextHolder.setAuthentication(...)
    → AuthController.me → AuthServiceImp.getCurrentUser("jane")
      → findByUsernameWithDetails → roles sorted, employeeId
      → 200 { username, email, roles:["ROLE_MANAGER"], employeeId:1 }
    → set({ user: { ... } })                    roles now in memory
→ navigate("/")
  → RequireAuth: token && user → renders <Outlet/> → Layout + DashboardPage
  → DashboardPage runs its 3 queries, now enabled: canManage === true

```

Failure path, all the way back

```

401 arrives → api() line 13: token is null, so the global logout branch is SKIPPED
→ response.ok === false
→ throw new Error(errorData.message) // "Invalid username or password"
→ catch (error) in handleSubmit
→ pushToast(error.message, "error") uiStore: toasts array grows
→ <Toasts/> re-renders (subscribed via useUIStore())
→ red box appears bottom-right, auto-removes after 4000 ms
→ finally { setSubmitting(false) } button re-enabled

```

TEXT

The round trip in one line

click → POST /api/auth/login → BCrypt match → sign JWT → 200 {token}
 → localStorage + Zustand → GET /api/auth/me → SecurityContext → 200 {user, roles}
 → navigate("/") → DashboardPage queries fire → manager UI renders

TEXT

3. Trace 2 - Register

Outbound

A. TRIGGER
 RegisterPage: username / password / email inputs
 → the hint "Email must match an existing employee" is a UI warning about the backend rule
 → click "Register"

B. REQUEST http.js:118
 POST /api/auth/register
 body: { "username": "jane",
 "password": "secret123",
 "email": "jane.smith@example.com" }
 no Authorization header (not logged in yet)

C. INBOUND
 /api/auth/** → permitAll → filter passes through anonymous

D. BUSINESS
 AuthController.register(@Valid @RequestBody RegisterRequest) (line 33)
 → @Valid runs Bean Validation FIRST, before the body executes:
 @NotBlank username → "Username is required" style errors
 @Size(min=6) password → "Password size must be minimum of 6"
 >Email email → "invalid email format"
 failure → MethodArgumentNotValidException (thrown before the method runs)

UserRegistrationFacade.register (line 24)

1. userService.existsByUsername("jane")
 → SELECT COUNT(*) FROM users WHERE UPPER(username) = UPPER('jane')
 → true → DuplicateResourceException (409)
2. employeeService.getEmployeeByEmail("jane.smith@example.com")
 → SELECT * FROM employees WHERE LOWER(email) = LOWER(?)
 → absent → ResourceNotFoundException (404)
 THIS is the gate that stops you claiming someone else's identity
3. roleService.getEmployeeRole()
 → SELECT * FROM roles WHERE name = 'ROLE_EMPLOYEE'
 → absent → ResourceNotFoundException (404)
 note: ALWAYS employee. no parameter, no way to ask for admin.
4. userService.createUser(username, rawPassword, role, employee)
 → existsByUsername AGAIN (defence in depth, service-level) (409)
 → user.setPassword(passwordEncoder.encode(raw)) BCrypt hash
 → user.setEnabled(true)
 → user.getRoles().add(role)
 → user.setEmployee(employee)

TEXT

```

→ userRepository.save(user)          INSERT INTO users ...
→ authService.generateToken(user)      same JWT as login

```

E. RESPONSE

```

201 Created
{ "token": " ... " }

```

Inbound

F. RETURN

```

identical to login from here on: setSession → loadUser → navigate("/")
and because the role is ROLE_EMPLOYEE, this time:
→ Layout nav renders ONLY "Attendance"
→ "/" DashboardPage renders, then: if (!canManage) return <Navigate to="/attendance" replace
/>
→ the employee lands on Attendance

```

The 404 that surprises people

If the email has no employee record the response is **404**, not 400. The backend treats "no employee with that email" as a missing resource. `api()` throws `Error("Employee Not Found")` and the toast shows exactly that. This is why the page warns you *before* you submit.

4. Trace 3 - Session restore on reload

No user action beyond pressing F5.

A. TRIGGER

```

the browser re-downloads index.html, main.jsx runs again
→ create() initialises authStore:
  token: localStorage.getItem("ems_token") → the OLD token, still valid
  user: null → roles are gone

```

B. REQUEST

```

App.jsx mounts the router
→ "/" matches RequireAuth
→ RequireAuth renders: token truthy, user null → "Loading ..."
→ useEffect: if (token && !user) loadUser()

```

C. INBOUND

```

GET /api/auth/me    Authorization: Bearer <old token>
→ JwtAuthenticationFilter
  header present, starts with "Bearer "
  → extractUsername: parseClaimsJws verifies HMAC + exp
    expired or tampered → throws → catch → continue ANONYMOUS
  → valid: loadUserByUsername (DB read)
  → isValid: subject matches AND not expired
  → SecurityContext populated
→ SecurityConfig: /api/auth/** is permitAll, so the filter chain does not

```

```

    reject early; the CONTROLLER requires an Authentication argument
    → Authentication is null → controller returns 401
    Authentication present → getCurrentUser(name)

D. BUSINESS
AuthServiceImp.getCurrentUser (line 53)
    → findByUsernameWithDetails(username)
        SELECT ... LEFT JOIN FETCH u.roles LEFT JOIN FETCH u.employee
    → roles.stream().map(Role::getName).sorted()
    → employeeId = user.getEmployee() ≠ null ? getId() : null
    → user.getEmail() → the helper that reads through to employee.email

E. RESPONSE
200 { "username":"jane", "email":"jane.smith@example.com",
      "roles":["ROLE_MANAGER"], "employeeId":1 }
or 401 { "message":"Unauthorized: invalid or missing token" }

F. RETURN
200 → set({ user }) → RequireAuth re-renders with token && user
    → <Outlet/> → Layout + the page → queries fire
401 → loadUser calls get().logout()
    → localStorage.removeItem("ems_token"); set({token:null,user:null})
    → RequireAuth: !token → <Navigate to="/login" replace />
network error → catch {} swallows it → stuck on "Loading ..." (deliberate)

```

Key insight: the token survives the reload from `localStorage`; the *identity* does not, and is re-proved against the server. That is why a deleted or downgraded account loses access on the next page load even with a valid token sitting in storage.

5. Trace 4 - Log out

The only flow with **no** backend request.

```

A. TRIGGER
click "Logout" in Layout.jsx

B. REQUEST      (none)

C. INBOUND      (none)

D. BUSINESS     (none)

E. RESPONSE     (none)

F. RETURN
handleLogout() Layout.jsx:29
  1. queryClient.clear()
    wipes every cached useQuery result, so the next user cannot see
    this user's data flash on screen
  2. logout()
    localStorage.removeItem("ems_token")

```

```

    set({ token:null, user:null })
3. navigate("/login")
    LoginPage renders; `if (token) return <Navigate to="/" replace />`
    does NOT fire because the token is now null

```

The important consequence: a JWT is stateless, so nothing was revoked server-side. A copy of the token would still work until it expires (24 h). See [07-security-model.md](#) section 6.

6. Trace 5 - Browse the employee list

Outbound

A. TRIGGER

```

nav click "Employees" (link has canManage: true, so an employee never sees it)
→ router matches /employees → EmployeesPage mounts
→ useState(0) page

```

B. REQUEST EmployeesPage:23 → http.js:32

```

canManage ≡ true and search ≡ ""
→ fetchEmployees(0)
→ api("/api/employees/paged?page=0&size=10")
    Authorization: Bearer <token>
    (no body, so no Content-Type)

```

C. INBOUND

```

→ JwtAuthenticationFilter populates the SecurityContext
→ anyRequest().authenticated() satisfied
→ @PreAuthorize("hasAnyRole('ADMIN','MANAGER')") on the CLASS
    employee → AccessDeniedException → 403, method body never runs
    manager → passes

```

D. BUSINESS

```

EmployeeController.getEmployeesPaged(Pageable pageable) (line 48)
→ Spring's PageableHandlerMethodArgumentResolver binds
    ?page=0&size=10 from the query string (no annotation needed)
→ employeeService.getAllEmployees(pageable)

EmployeeServiceImp.getAllEmployees(Pageable) (line 82)
→ employeeRepository.findAll(pageable)
    SELECT e.* FROM employees e LIMIT ?, ?
    SELECT COUNT(*) FROM employees (for totalElements)
→ each row becomes an Employee entity
→ .map(employeeMapper::toDtoResponse)
    MapStruct-generated code copies id, firstName, ... and maps
    department → DepartmentSummary{id, name}
    (legal here: the class-level @Transactional is still open, so the
    LAZY department can be read)

```

E. RESPONSE

```

200
{

```

```

"content": [ { "id":1, "firstName":"Jane", "lastName":"Smith",
               "email":"jane.smith@example.com", "phone":"555-0101",
               "hireDate":"2023-05-01", "salary":65000,
               "department": { "id":1, "name":"Engineering" } } ],
"totalElements": 5, "totalPages": 1, "size": 10, "number": 0, ...
}

```

Inbound

F. RETURN

TanStack Query receives the Page under key `["employees", 0, ""]`

- `isLoading` → `false`, `isError` → `false`
- `const rows = search.trim() ? (data ?? []) : (data?.content ?? [])` // Page → content
- `<tbody>` maps rows into `<tr>`, one per employee
- salary rendered through `Intl.NumberFormat` (USD), `"-"` when null
- pager: `"Page 1 of 1"`, both buttons disabled because page `≡ 0` and `page + 1 ≥ totalPages`

NEXT PAGE:

- click `"Next"` → `setPage(p ⇒ p+1)` → `page = 1`
- `queryKey` changes to `["employees", 1, ""]` → NEW network request
- GET `/api/employees/paged?page=1&size=10`
- `placeholderData` is not used here, so rows briefly empty while loading

TEXT

7. Trace 6 - Search employees by name

A. TRIGGER

type into the search box → `setSearchInput` (NO request fires)
 press `"Search"` (form submit) → `handleSubmit`
`setSearch(searchInput); setPage(0)`
 two states on purpose: typing must not fetch per keystroke

B. REQUEST

http.js:38

`search.trim()` is now non-empty → `searchEmployee(search.trim())`
 → GET `/api/employees/search?name=jane` Authorization: Bearer `<token>`
 (note: the name is interpolated raw, no `encodeURIComponent`)

C. INBOUND

same as trace 5: authenticate, then the class-level `@PreAuthorize`

D. BUSINESS

`EmployeeController.searchEmployees(@RequestParam String name)` (line 54)
 → `@RequestParam: absent 'name'` → 400 (`MissingServletRequestParameterException` is unhandled, so it lands in the generic 500 branch)
 → `getEmployeesByFullNameContaining("jane")`

`EmployeeServiceImp` (line 69)

→ `employeeRepository.findByFullNameContainingWithDepartment("jane")`
`SELECT DISTINCT e FROM employees e`
`LEFT JOIN FETCH e.department d`

TEXT

```

WHERE LOWER(CONCAT(e.firstName, ' ', e.lastName))
      LIKE LOWER(CONCAT('%', :fullName, '%'))
DISTINCT → collapses join duplication
LEFT JOIN FETCH → department arrives in the same query,
                  so the lazy getter is safe to read
→ .map(employeeMapper::toDtoResponse)

```

E. RESPONSE

```
200 [ { "id":1, "firstName":"Jane", ... } ]    a bare List, NOT a Page
```

F. RETURN

```

cached under ["employees", 0, "jane"]
→ const rows = search.trim() ? (data ?? []) : (data?.content ?? [])
    ^^^^^^^^^^^^^^^^^ a bare array this time
→ pager is hidden: {!isSearching && (...)}

```

The two response shapes (array vs `Page`) are the subtle part of this flow, and the ternary on `search.trim()` is the only thing keeping them apart.

8. Trace 7 - Create an employee

The longest flow: **two** requests before the write, then a write, then refetches.

A. TRIGGER

```

click "Add Employee" → setModal({ mode:"create" })
→ EmployeeFormModal mounts; useQuery fetches the department dropdown
→ fill first/last/email/phone/salary/hireDate, pick a department
→ click "Create" → handleSubmit → mutate({ id: undefined, employeeFormData: form })

```

B. REQUESTS (in order)

```

B1 GET /api/departments?page=0&size=100    (dropdown, on mount)
B2 POST /api/employees
   Content-Type: application/json
   Authorization: Bearer <token>
   body: { firstName, lastName, email, phone, salary,
          hireDate, departmentName }      ← department NAME, not id

```

C. INBOUND (B2)

```

JWT filter → SecurityContext → class @PreAuthorize
→ @Valid on EmployeeCreateRequest:
   @NotBlank firstName / lastName / email / phone / departmentName
   @Email email
   @DecimalMin(value="0.0", inclusive=false) salary → 0 is REJECTED
   hireDate no constraint (a bad string fails Jackson → 400)
failure → MethodArgumentNotValidException → 400 + fieldErrors map

```

D. BUSINESS

```

EmployeeServiceImp.createEmployee    (line 32)
1. existsByEmailIgnoreCase(email.trim())
   → SELECT COUNT(*) FROM employees WHERE LOWER(email) = LOWER(?)
   → true → DuplicateResourceException    (409)
2. employeeMapper.toEntity(request)

```

```

compile-time-generated copy; `department` is @Mapping(ignore = true)
because the DTO has a departmentName STRING, not a Department
3. departmentRepository.findByNameIgnoreCase(departmentName.trim())
   → SELECT * FROM departments WHERE LOWER(name) = LOWER(?)
   → absent → ResourceNotFoundException (404)
4. employee.setDepartment(department) ← the relationship, set here
5. employeeRepository.save(employee)   INSERT INTO employees ...
6. employeeMapper.toDtoResponse(saved)

```

E. RESPONSE

201 Created

```
{ "id":6, "firstName":" ... ", "department":{"id":2, "name":"HR" } }
```

F. RETURN

useMutation onSuccess:

```

pushToast("Employee Created")           green toast, auto-hides at 4 s
queryClient.invalidateQueries({queryKey:["employees"]})
  prefix match → EVERY ["employees", ...] entry is stale
  → EmployeesPage's active query refetches (a 3rd request)
  → the new row appears; the user stays on whatever page they were on
onClose()                               modal unmounts, form state gone

```

Errors return the same way: `onError` -> `pushToast(error.message, "error")`, and the modal **stays open** so the values are still there to fix.

9. Trace 8 - Update an employee

A. TRIGGER

```

click "Edit" on a row → setModal({ mode:"edit", employee: emp })
→ the modal pre-fills from `employee?.x || ""`; salary uses `?? ""`
  so a real 0 is preserved
→ hireDate input is NOT rendered (mode ≡ "create" only)
→ click "Save" → mutate({ id: employee.id, employeeFormData: form })

```

B. REQUEST

PATCH /api/employees/6

Content-Type: application/json

```
body: { firstName, lastName, email, phone, salary, departmentName }
every field present, because the form always sends the whole object
```

C. INBOUND

```

authenticate → @PreAuthorize (class) → @Valid on EmployeeUpdateRequest
→ only @Email and @DecimalMin apply, and both are skipped for null values

```

D. BUSINESS

EmployeeServiceImp.updateEmployee (line 89)

```

1. employeeRepository.findById(6)
   → absent → ResourceNotFoundException (404)
   → present → MANAGED entity (attached to the session)
2. email uniqueness, THREE conditions ANDed:
   request.getEmail() ≠ null
   && !employee.getEmail().equalsIgnoreCase(request.getEmail())

```

TEXT

```

    && employeeRepository.existsByEmailIgnoreCase(request.getEmail())
    the middle term is why editing only the phone does not 409 on yourself
3. department only if departmentName ≠ null
    → unknown name → ResourceNotFoundException (404)
4. employeeMapper.updateEmployee(request, employee)
    NullValuePropertyMappingStrategy.IGNORE → nulls are SKIPPED
    @MappingTarget → mutates the managed entity in place
5. if (department ≠ null) employee.setDepartment(department)
6. return employeeMapper.toDtoResponse(employee)
    NO employeeRepository.save() CALL
    @Transactional + dirty checking flushes the changed columns on commit

E. RESPONSE
200 with the updated employee (the same entity, re-mapped after the flush)

F. RETURN
pushToast("Employee Updated") → invalidateQueries(["employees"]) → refetch
→ onClose()

```

The absence of `save()` is the detail worth being able to explain: the entity was loaded inside a transaction, so Hibernate compares it to the row at commit time and issues `UPDATE`s only for what actually changed.

10. Trace 9 - Delete an employee

TEXT

```

A. TRIGGER
click "Delete" → handleDelete(id)
→ window.confirm("Delete this employee?") blocking browser dialog

B. REQUEST
DELETE /api/employees/6
no body → no Content-Type header (api() only sets it when options.body)

C. INBOUND
authenticate → @PreAuthorize (class)

D. BUSINESS
EmployeeServiceImp.deleteEmployee (line 119)
1. employeeRepository.findById(6) → 404 if missing
2. employeeRepository.delete(employee)
    → the Employee.attendanceList mapping is
       cascade = ALL, orphanRemoval = true
    → Hibernate issues DELETE for each attendance row first,
       then DELETE FROM employees WHERE id = ?
    → if a 'users' row still references this employee_id, the FK
       rejects it → DataIntegrityViolationException → 409

E. RESPONSE
204 No Content, empty body
→ api() line 27: status ≡ 204 → return null

F. RETURN

```

```
deleteMutate onSuccess:
  pushToast("Employee Deleted")
  invalidateQueries(["employees"]) → refetch → row is gone
```

11. Trace 10 - Browse departments

Identical in shape to trace 5, with three differences worth naming:

```
B. GET /api/departments/paginated?page=0&size=10 (note: "paginated",
                                                    employees use "paged")

D. DepartmentServiceImp.getAllDepartments(pageable)
  → findAll(pageable) → .map(toDtoResponse)
  no LEFT JOIN FETCH needed: DepartmentResponse has no employee field,
  so MapStruct never touches the LAZY employeeList

E. 200 Page<DepartmentResponse> { id, name, description }

F. const rows = data?.content ?? []
```

TEXT

`DepartmentsPage` also carries a second piece of state, `details`, used by trace 12.

12. Trace 11 - Create / update / delete a department

Create

```
A. click "Add Department" → setModal({mode:"create"}) → fill name + description
B. POST /api/departments { name, description }
C. @PreAuthorize (class) → @Valid:
   name      @NotBlank + @Size(max=100)
   description @NotBlank + @Size(max=255)
D. DepartmentServiceImp.createDepartment (line 31)
   1. existsByNameIgnoreCase(name) → DuplicateResourceException (409)
   2. departmentMapper.toEntity(createReq)
   3. departmentRepository.save(department) INSERT
E. 201 { "id":4, "name":"Legal", "description":" ... " }
F. pushToast("Department created") → invalidateQueries(["departments"]) → onClose
```

TEXT

Update

```
A. click "Edit" → setModal({mode:"edit", department: dep}) → change description
B. PUT /api/departments/4 { name:"Legal", description:"New text" }
   (PUT here, PATCH for employees; both behave partially because the mapper
   IGNORE strategy skips nulls)
C. @PreAuthorize → @Valid (@Size only)
```

TEXT

```

D. DepartmentServiceImp.updateDepartment (line 81)
  1. findById(4) → 404 if missing
  2. validateDepartmentUpdate(updateReq, department) (private helper, line 109)
    a. if name ≠ null:
      if name.trim().isEmpty() → InvalidInputException (400)
      if !current.equalsIgnoreCase(new) && existsByNameIgnoreCase(new)
        → DuplicateResourceException (409)
    b. if description ≠ null: NO VALIDATION AT ALL
      "" passes @Size, so a blank description can overwrite the
      nullable = false column. Known gap.
  3. departmentMapper.updateDepartment(updateReq, department)
    @MappingTarget mutates the managed entity; nulls skipped
  4. return toDtoResponse(department) NO save() call
E. 200 the updated department
F. showToast("Department updated") → invalidateQueries(["departments"]) → onClose

```

Delete

```

A. click "Delete" → window.confirm("Delete this Department")
B. DELETE /api/departments/4 no body
C. @PreAuthorize (class)
D. DepartmentServiceImp.deleteDepartment (line 95)
  1. findById(4) → 404
  2. if (!department.getEmployeeList().isEmpty())
    → InvalidInputException("Cannot delete department with assigned employees")
    → 400
    BUSINESS RULE, not a DB rule: employees.department_id is nullable,
    so the database would happily orphan the employees
  3. departmentRepository.delete(department)
E. 204 No Content (ResponseEntity.noContent().build())
F. showToast("Department Deleted") → invalidateQueries(["departments"])

```

TEXT

13. Trace 12 - View a department with its employees

The clearest example of a flow that starts from **already-loaded data** and then fetches more.

```

A. TRIGGER
  the row's name and description are ALREADY in memory (from trace 10)
  click "View" → setDetails(dep)
  → DepartmentDetailsModal mounts with department={dep}

B. REQUEST http.js:67
  useQuery({ queryKey: ["department-employees", department.id],
    queryFn: () ⇒ fetchDepartmentEmployees(department.id) })
  → GET /api/departments/4/employees
  note the id in the URL comes from the department object already on screen;
  the modal makes its own request rather than reusing page data

C. INBOUND
  authenticate → class @PreAuthorize on DepartmentController

```

TEXT

(the whole controller is manager-only because this response carries salary)

D. BUSINESS

```
DepartmentController.getDepartmentByIdWithEmployees (line 56)
  → DepartmentServiceImp.getDepartmentByIdWithEmployees (line 54)
    → departmentRepository.findWithEmployeesById(4)
      SELECT DISTINCT d FROM departments d
      LEFT JOIN FETCH d.employeeList e WHERE d.id = 4
      DISTINCT → collapses the row multiplication
      LEFT JOIN FETCH → employeeList arrives WITH the department,
      so toDtoResponseDetails can read the LAZY list safely
    → departmentMapper.toDtoResponseDetails(department)
      maps Department → DepartmentDetailsResponse
      and each Employee → nested EmployeeBrief
      (shared field names are matched; attendanceList and the inverse
      user link are simply not copied)
```

E. RESPONSE

```
200
{ "id":4, "name":"Legal", "description":" ... ",
  "employeeList":[ { "id":7, "firstName":" ... ", "salary":52000 } ] }
```

F. RETURN

```
cached under ["department-employees", 4]
→ const employees = data?.employeeList || []
→ table renders name / email / salary (Intl currency format, "-" if null)
→ "No employees" when the list is empty; error.message on failure
→ Close → setDetails(null) → modal unmounts
→ CACHING CONSEQUENCE: reopening the same department serves from cache with
  no new request unless the key changed
```

14. Trace 13 - Check in

Manager path

A. TRIGGER

```
the "Employee" <select> is populated from ["employee-list"] (GET /api/employees)
→ setClockEmployeeId("6")
→ effectiveClockEmployeeId = clockEmployeeId (derived, no effect)
→ the button renders "Check In" because todayRow?.checkInTime is falsy
```

B. REQUEST

http.js:100

```
POST /api/attendance/check-in?employeeId=6
NO body → api() sends no Content-Type
Authorization: Bearer <token>
```

C. INBOUND

```
JWT filter → SecurityContext
→ @PreAuthorize SpEL:
  hasAnyRole('ADMIN','MANAGER') TRUE → short-circuits, allowed
  (the ownership half is never even evaluated)
→ a plain ROLE_EMPLOYEE caller:
```

TEXT

```

hasAnyRole(...) FALSE
hasRole('EMPLOYEE') TRUE
authentication.principal.employee not null
#employeeId ("6") == own employee.id FALSE if they picked someone else
→ 403, method body never runs

```

D. BUSINESS

```

AttendanceServiceImp.checkIn(6) (line 74)
1. employeeRepository.findById(6) → InvalidInputException (400) if absent
2. LocalDate today = LocalDate.now() SERVER clock, not the browser's
3. attendanceRepository.findByIdAndDate(6, today)
   SELECT * FROM attendances WHERE employee_id = 6 AND date = ?
   found (e.g. a seeded ABSENT row)
   → .orElseGet() is NOT called, the existing entity is reused
   not found
   → .orElseGet() creates Attendance{employee, date=today}
4. if (attendance.getCheckIn() ≠ null)
   → InvalidInputException("Employee already checked in today") (400)
5. setCheckIn(LocalDate.now()) column is TIME, not a timestamp
   setCheckOut(null) explicitly clears a previous state
   setTotalHours(null)
   setStatus(INCOMPLETE) the only place INCOMPLETE is set
6. attendanceRepository.save(attendance)
   INSERT (new row) or UPDATE (reused row)
7. attendanceMapper.toDtoResponse(saved)
   checkInTime ← LocalDateTime.of(date, checkIn)
   employeeName ← employee.fullName ("Jane Smith")

```

E. RESPONSE

```

200
{ "id":71, "employeeId":6, "employeeName":"Maria Garcia",
  "date":"2026-01-15",
  "checkInTime":"2026-01-15T09:03:11", "checkOutTime":null,
  "status":"INCOMPLETE" }

```

Inbound

F. RETURN

```

onSuccess(data):
  pushToast("Checked in at 09:03") formatTime = value.slice(11,16)
  invalidateAttendance() → THREE refetches:
    invalidateQueries(["attendance"]) the main table
    invalidateQueries(["attendance-today"]) the clock card button state
    invalidateQueries(["dashboard-attendance"]) the dashboard counters
  each refetch re-runs its own A→F trace

the clock card now renders the second branch:
todayRow.checkInTime && !todayRow.checkOutTime
→ "Checked in at 09:03" + a "Check Out" button

```

TEXT

Employee path

The employee cannot reach the dropdown, and `effectiveClockEmployeeId` comes from the token:

```
effectiveClockEmployeeId = canManage ? clockEmployeeId
                                   : String(user?.employeeId ?? "")
    user.employeeId was supplied by GET /api/auth/me
    the UI shows their own username instead of a <select>
→ POST /api/attendance/check-in?employeeId=<their own id>
→ SpEL: hasRole('EMPLOYEE') TRUE
    principal.employee ≠ null      TRUE (registration guarantees this)
    #employeeId == principal.employee.id TRUE
→ identical service code from here on
```

TEXT

Even if an employee hand-edited the URL to another id, the SpEL comparison would fail and return **403**.

15. Trace 14 - Check out

```
A. TRIGGER
the card renders "Check Out" (checked in, not checked out)
→ checkOutMutate(effectiveClockEmployeeId)

B. REQUEST
POST /api/attendance/check-out?employeeId=6      no body
the SAME SpEL ownership rule as check-in

C. INBOUND
authenticate → SpEL (identical to check-in)

D. BUSINESS
AttendanceServiceImp.checkOut(6)                  (line 102)
1. employeeRepository.existsById(6) → 400 if absent
   (existsById, not findById: the Employee object is not needed here)
2. attendanceRepository.findByIdAndDate(6, today)
   absent → InvalidInputException("Employee has not checked in today") (400)
3. if (attendance.getCheckIn() == null)
   → InvalidInputException("Employee has not checked in today") (400)
   two guards for two situations: no row at all, or a seeded ABSENT row
4. LocalTime checkOut = LocalTime.now()
5. double totalHours = Math.round(
    Duration.between(checkIn, checkOut).toMinutes() / 60.0 * 100.0)
    / 100.0
   minutes → hours via 60.0 (forces float division)
   → round to 2 decimals
6. attendance.setStatus(computeStatus(checkIn, checkOut, totalHours))
   totalHours < 4.0      → HALF_DAY (checked FIRST)
   checkIn.isAfter(09:00) → LATE
   otherwise            → PRESENT
7. attendanceRepository.save(attendance) UPDATE

E. RESPONSE
200 { ..., "checkOutTime":"2026-01-15T17:34:02", "status":"PRESENT" }
note totalHours is NOT in the response - written to the DB, never returned

F. RETURN
```

TEXT

```
onSuccess(data):
    showToast("Checked out at 17:34 - present")
    statusLabel(status) = status.replace(/_/g, " ").toLowerCase()
    invalidateAttendance() → same three refetches
the card now renders the third branch:
    todayRow.checkInTime && todayRow.checkOutTime
    → "Done: present"
    → the Check Out button is GONE for the day
```

`checkOut` has **no** "already checked out" guard. A second call simply overwrites `checkOut` and recomputes - the button is hidden in the UI, but the endpoint will accept a repeat.

16. Trace 15 - Manager searches attendance

TEXT

```
A. TRIGGER
choose Department / Employee / Status / From / To
→ handleChange → setFilters(...) NO request yet
→ click "Apply" → handleApply
    setApplied({ ... nulls-for-empty-strings ... }); setPage(0)
"filters" = the inputs, "applied" = what was submitted; only 'applied'
is in the query key, so nothing is fetched until Apply

B. REQUEST http.js:87
POST /api/attendance/search?page=0&size=10
Content-Type: application/json
body: { "employeeId":null, "departmentId":2, "status":"LATE",
        "startDate":"2026-01-01","endDate":"2026-01-15" }
Reset → emptyApplied → all nulls → empty filter set

C. INBOUND
authenticate
→ @PreAuthorize("hasAnyRole('ADMIN','MANAGER')") method-level
    an employee → 403

D. BUSINESS
AttendanceController.searchAttendance(@RequestBody(required=false) req,
                                     Pageable pageable) (line 35)
→ req may be null (no body, or a literal 'null' body)
→ AttendanceServiceImp.searchAttendance (line 41)
    @Transactional(readOnly = true) LOAD-BEARING, see below

1. criteria = (req == null) ? new AttendanceSearchReq(null x5) : req
   normalising null is what stopped the NPE: without it,
   criteria.employeeId() would dereference null → 500

2. build a LIST of Specifications, one per non-null filter:
   employeeId ≠ null → hasEmployeeId → employee.id = ?
   departmentId ≠ null → hasDepartmentId → employee.department.id = ?
   status ≠ null → hasStatus → status = ?
   startDate ≠ null AND endDate ≠ null → dateBetween → date BETWEEN ? AND ?
   ^ BOTH bounds are required; one alone is silently ignored
```

```

3. Specification.allOf(filters)
   NOT Specification.where(null): that is forbidden in Spring Data JPA 4
   and throws "Specification must not be null". allOf([]) matches
   everything, which is what an empty filter set needs.

4. attendanceRepository.findAll(spec, pageable)
   SELECT a.* FROM attendances a
   [JOIN employees e ...] [JOIN departments d ...]
   WHERE <only the provided predicates>
   LIMIT ?, ?
   SELECT COUNT(*) ... same WHERE

5. .map(attendanceMapper::toDtoResponse)
   reads attendance.getEmployee().getFullName() on a LAZY association.
   That is safe ONLY because readOnly = true keeps the Hibernate
   session open for the whole method. Without it: LazyInitializationException
   → generic 500 handler.

```

E. RESPONSE

```

200
{ "content": [ { "id": 71, "employeeId": 6, "employeeName": "Maria Garcia",
                "date": "2026-01-14", "checkInTime": "... T09:41:00",
                "checkOutTime": "... T17:30:00", "status": "LATE" } ],
  "totalElements": 45, "totalPages": 5, "size": 10, "number": 0 }

```

Inbound

F. RETURN

```

key ["attendance", "search", applied, page]
→ placeholderData: prev keeps the old rows visible while refetching
→ isApplying = isFetching && !isLoading → the Apply button says
   "Applying ..." and is disabled, but the table does NOT blank out
→ rows = data?.content ?? []
→ the Employee column renders (canManage)
→ "Next" → setPage(p ⇒ p+1) → page 2 → new request with page=2

```

TEXT

17. Trace 16 - An employee views their own attendance

The read path with **no** scope parameter at all.

A. TRIGGER

```

nav click "Attendance" (the only link without canManage)
→ AttendancePage mounts
→ canManage ≡ false, so:
   effectiveClockEmployeeId = String(user?.employeeId ?? "")
   the filter <form> is NOT rendered at all ({canManage && (...)})
   the Employee column header is NOT rendered
   the heading reads "My Clock In / Out" and the username is shown
   as plain text instead of a <select>

```

TEXT

```

B. REQUEST http.js:94
  TWO requests fire on mount:
  B1 GET /api/attendance/me?page=0&size=10
    (page, size always; startDate/endDate appended only when truthy)
    Authorization: Bearer <token>
    *** no employeeId anywhere - there is nothing to tamper with ***
  B2 GET /api/attendance/me?page=0&size=1&startDate=<today>&endDate=<today>
    the "today" query that drives the Check In / Check Out button

C. INBOUND
  JWT filter → SecurityContext
  → @PreAuthorize("hasRole('EMPLOYEE')") method-level
    ROLE_EMPLOYEE → allowed
    ADMIN / MANAGER → 403 (they must use /search instead)

D. BUSINESS
  AttendanceController.myAttendance(
    @AuthenticationPrincipal User user, ← SecurityContext
    @RequestParam(required=false) @DateTimeFormat(iso=DATE) startDate,
    @RequestParam(required=false) @DateTimeFormat(iso=DATE) endDate,
    Pageable pageable) (line 46)

    @AuthenticationPrincipal == authentication.getPrincipal() cast to User.
    Because User implements UserDetails, the principal IS the entity.

    → AttendanceSearchReq req = new AttendanceSearchReq(
      user.getEmployee().getId(), ← FROM THE TOKEN, never the request
      null, ← no department filter
      startDate, endDate,
      null) ← no status filter
    → attendanceService.searchAttendance(req, pageable)
      the SAME service method as /search, so the filtering logic is identical

E. RESPONSE
  200 Page<AttendanceResponse> containing ONLY this employee's rows
  (verified against a running server: 17 rows for the test employee, while the
  same data via /search as a manager returned 45 organisation-wide rows)

F. RETURN
  main table → key ["attendance", "me", applied, page]
    (the role string "me" is part of the key, so employee and
    manager caches can never collide)
  clock card → key ["attendance-today", "me", <id>]
  → rows render with 4 columns (no Employee column)
  → the pager works the same way

```

This is the security design in its clearest form. The scoping is not "validate that the employee asked for their own id" - it is "never accept an id, read it from the token". There is no request the client can craft to widen it.

18. Trace 17 - The dashboard

One user goal, **three** independent requests, **zero** statistics endpoints.

TEXT

A. TRIGGER

```
nav click "Dashboard" (or land on "/" after login)
→ three useQuery calls are declared with enabled: canManage
→ an employee: the queries never fire, and
  if (!canManage) return <Navigate to="/attendance" replace />
```

B. REQUESTS (parallel)

```
B1 GET /api/employees → List, ALL rows
B2 GET /api/departments?page=0&size=100 → Page, up to 100
B3 POST /api/attendance/search?page=0&size=1000
  body {} ← an EMPTY filter object, so no Specification is added
           and every attendance row is returned (size 1000 to avoid
           paging; totalElements comes from the real COUNT query)
```

C. INBOUND

```
all three: JWT filter → class @PreAuthorize on their controllers
```

D. BUSINESS

```
plain repository reads, no rules
(B3 is the same searchAttendance path as trace 15, with an empty filter list)
```

E. RESPONSE

```
B1 200 [ EmployeeResponse x N ]
B2 200 Page { content:[ DepartmentResponse x M ], totalElements: M }
B3 200 Page { content:[ AttendanceResponse x K ], totalElements: K }
```

F. RETURN - all aggregation happens IN THE BROWSER

```
isLoading = attLoading || empLoading || deptLoading → one "Loading ..."
isError = attError || empError || deptError
→ Total Employees = employeeList.length
→ Departments = departmentList.length
                  (departmentList = departments.content)
→ Attendance Records = attendance.totalElements ← from the Page,
                  NOT content.length
→ status chips: attendanceRow.forEach(att ⇒
  totalStatusCounts[att.status]++ guarded by !== undefined)
→ Recent Attendance: [... attendanceRow]
  .sort((a,b) ⇒ a.date < b.date ? 1 : -1) descending by ISO string
  .slice(0, 8)
→ Employees by Department: departmentList.map(dep ⇒ ({
  ... dep,
  count: employeeList.filter(e ⇒ e.department?.id === dep.id).length
})))
→ statusStyles maps each status to Tailwind classes
→ statusLabel = status.replace(/_/g, " ").toLowerCase()
```

```
sort note: `[... attendanceRow]` copies first, so the cached array is not
mutated; ISO date strings sort correctly with plain '<'.
```

Consistency caveat worth stating out loud: the three requests are independent, so a clock-in landing between B1 and B3 can make the numbers momentarily disagree. No transaction spans them.

19. Trace 18 - A request that is refused

The most common "nothing happens" case, traced completely.

A. TRIGGER

an employee opens /employees (e.g. by editing the URL)

B. REQUEST

EmployeesPage mounts

→ useQuery({ enabled: canManage }) → NO REQUEST IS SENT

→ then: if (!canManage) return <Navigate to="/attendance" replace />

so the guard runs before any network call. The redirect happens after the hooks on purpose: returning before them would change the hook order between renders and break React's rules of hooks.

C. INBOUND

(nothing arrives; there is no request)

And if the employee DID reach the API directly with a valid token, e.g. curl:

JWT filter populates the context (they ARE authenticated)

→ anyRequest().authenticated() passes

→ @PreAuthorize("hasAnyRole('ADMIN','MANAGER')") FAILS

→ AccessDeniedException

→ GlobalExceptionHandler.handleAccessDenied → 403

If the token is missing or expired instead:

→ the filter leaves the context anonymous

→ anyRequest().authenticated() FAILS

→ SecurityConfig.authenticationEntryPoint

→ 401 {"status":401,"message":"Unauthorized: invalid or missing token"}

E. RESPONSE

frontend path: <Navigate to="/attendance" replace /> (nothing was fetched)

curl path: 403 { "status":403,
"message":"Forbidden: insufficient permissions" }

F. RETURN

frontend: the router swaps in AttendancePage

replace → the /employees entry is REPLACED in history, so Back
does not bounce straight into the guard again

api(): a 403 is not a 401, so the global logout branch does NOT run;
the user stays signed in and the error surfaces as a toast

Note the distinction: the global 401 handler logs you out; a 403 never does. Getting that backwards is a common source of "I got logged out randomly" bugs.

20. Every trace at a glance

#	Flow	Requests	Deciding backend rule
1	Log in	2 (<code>login</code> , <code>me</code>)	BCrypt match, then JWT signing
2	Register	2 (<code>register</code> , <code>me</code>)	email must exist; role is always EMPLOYEE
3	Session restore	1 (<code>me</code>)	token signature + expiry, roles re-read from DB
4	Log out	0	cache cleared, token removed, nothing revoked
5	Employee list	1 per page	class <code>@PreAuthorize</code>
6	Employee search	1	class <code>@PreAuthorize</code> ; <code>LOWER(CONCAT(...)) LIKE</code>
7	Create employee	2 + refetch	email uniqueness, department name must exist
8	Update employee	1 + refetch	3-term email check, mapper IGNORE, dirty checking
9	Delete employee	1 + refetch	cascade deletes attendance; FK on users may 409
10	Department list	1 per page	class <code>@PreAuthorize</code>
11a	Create department	1 + refetch	name uniqueness (409)
11b	Update department	1 + refetch	helper: empty name 400, changed name 409
11c	Delete department	1	refuses if employees assigned (400)
12	Department details	1	<code>LEFT JOIN FETCH employeeList</code>
13	Check in	1 + 3 refetches	SpEL ownership; reuse-or-create row; <code>INCOMPLETE</code>
14	Check out	1 + 3 refetches	two guards; hours -> HALF_DAY/LATE/PRESENT
15	Attendance search	1 per page/filters	<code>allof</code> specifications; <code>readOnly</code> tx is load-bearing
16	My attendance	2 on mount	id from <code>@AuthenticationPrincipal</code> , not the request
17	Dashboard	3 in parallel	counts aggregated client-side
18	Refused request	0 or 1	403 via <code>@PreAuthorize</code> , 401 via entry point

21. The five questions to ask about any flow

If you can answer these for a flow you can explain it:

- 1. What triggered it?** A click, a form submit, a mount, a page reload, or a cache invalidation from another flow.

2. **What went on the wire?** Exact method, URL, headers, body. Which of them came from the token and which from user input?
3. **Where was it rejected or allowed?** The filter chain for authentication, then `@PreAuthorize` for authorization. Name the exact annotation.
4. **Which layer owned the rule?** Business rule in the service, uniqueness in the service plus the DB constraint, shape validation in the DTO, SQL shape in the repository.
5. **What came back, and what changed on screen?** Status code, body, which query key was written or invalidated, whether a toast fired, whether anything navigated.

Answering those five for all 18 flows is the whole application.